Appendix B 解答

GB-Q01

1. Symbol addresses：

Symbol	Address
`main`	`0x00400000`
`loop`	`0x00400004`
`done`	`0x00400010`
`func`	`0x00400018`
`table`	`0x10010000`
`count`	`0x10010004`
`str`	`0x10010008`

2. `beq` 在 `0x00400008`。PC-relative base 是 $PC + 4 = 0x0040000c$，target `loop=0x00400004`：

$$(0x00400004 - 0x0040000c)/4 = -2 = 0xfffe_{16}.$$

3. J-format target field 是 `(target >> 2) & 0x03ffffff`：

- `j done`：$0x00400010/4 = 0x00100004$。
- `jal func`：$0x00400018/4 = 0x00100006$。

4. `.align 2` 對齊到 4 bytes；`table` 4 bytes、`count` 4 bytes、`"Hi"` 加 terminator 是 3 bytes。最後的 NUL 位於 `0x1001000a`。

GB-Q02

1. `jal external` 的 final target field：

$$0x00402000 \gg 2 = 0x00100800.$$

2. `jal` opcode 3 放在 bits 31:26：

$$(3 \ll 26) \mid 0x00100800 = 0x0c100800.$$

3. $0x12345678$ 的 unsigned low half 是 `0x5678`，小於 `0x8000`，所以 high half 不需 low-sign carry adjustment：

- `lui` immediate `0x1234`。
- `ori` immediate `0x5678`。

Appendix B 解答 （續）

兩種組合須分清 immediate 的延伸方式。`lui/ori` 使用原始 upper half $H = (V \gg 16)\ \&\ 0xffff$，因為 `ori` 的 low half 零延伸。`lui/addiu` 使用 $H = ((V + 0x8000) \gg 16)\ \&\ 0xffff$，再加上 sign-extended low half。例 $V = 0x1234abcd$：前者用 `0x1234` 與 `0xabcd`；後者用 `0x1235` 與 signed low -21555，兩者都還原同一個 32-bit 值。不能把帶進位的 upper half 和 `ori` 混用。

規格依據為 MIPS32 Instruction Set，LUI、ORI、ADDIU 與 branch restrictions。本題在 branch delay slot 放 `nop`，避免將另一控制轉移指令放入 delay slot。

4. Object 可在 instruction field 放 placeholder/addend 與 relocation type。Linker 根據 final symbol value、PC 與 relocation 的 bit layout 改寫該 field；metadata 也指定 signed/unsigned、高位截斷或 low/high-pair semantics。只有 numeric placeholder 而沒有 relocation type，linker 無法安全修正。

GB-Q03

1. P 的 text start 是 `0x00400000`。Q 接在 P 後：

$$0x00400000 + 192 = 0x004000c0.$$

2. P data start `0x10010000`，長 16；Q data start `0x10010010`。
3. `0x10010010` 已整除 8，不需額外 padding。Q data 長 24 bytes，因此 final end 是

$$0x10010010 + 0x18 = 0x10010028.$$

4. 兩個 strong global definitions 通常造成 duplicate symbol error；linker 不能以 object 順序任選而默默改變程式語意。Weak/common binding 依格式有 `weak` 與 common merging rules，但仍要檢查 resolved binding。Static local symbols 可在各自 object 共存。

GB-Q04

1. Calls for n=3：3、2、1、0，共 4 frames。總 stack 是

$$4 \times 16 = 64 \text{ bytes}.$$

Appendix B 解答 （續）

2. 每層 current `$sp` 起，`ra` 在 `12($sp)`、`a0` 在 `8($sp)`，relative offsets 不變；absolute addresses 隨各層 `$sp` 改變。Frame 間相隔 16 bytes。
3. Callee 由 inner 到 outer 執行 epilogue：先 load saved registers/`ra`，再執行 `addiu $sp,$sp,16`。若先移動`$sp`卻仍用舊offset，`lw $ra,12($sp)`會讀錯位置；並非舊frame絕對不可達，改用`-4($sp)`或暫存舊base仍可讀取。本題標準epilogue先restore再回復sp，以免混用offset。
4. Function ABI 只要求 caller/callee-saved registers contract；interrupt 可在任意 time 發生，hardware/exception entry 必須保存足以恢復 interrupted context 的 PC/status/registers。Handler 使用 application registers 時要 save/restore，否則會改變 application's architectural state。兩者都可使用stack，但exception vector及privilege切換並不是一般的 `jal` 呼叫語意。

GB-Q05

1. Bytes：

- Big-endian：`de ad be ef`。
- Little-endian：`ef be ad de`。

2. Address `0x102` 不是 4-byte aligned。Strict MIPS `lw` 可能發 alignment exception；有些 implementation 只允許 aligned access，軟體需拆成 byte loads 或使用 platform support。
3. Big-endian `0xbeef` 從 `0x100` 起是 bytes `be ef`，`lhu` 得 `0xbeef`。
4. Host memory dump 顯示 host endianness、padding、alignment 與 compiler packed-layout decisions；wire protocol 規定 byte order、field widths 與 serialization。Sender/receiver 必須用同一 format 與 validation，不能傳 native struct 假設 binary layout。

Appendix B 解答 （續）

GB-Q06

1. Teaching convention reads service number from `$v0`，arguments from `$a0..`，一般有回傳值的service常以`$v0`回傳；print-int本身沒有指定result，不能把原本service number當成計算結果。Specific SPIM services可能使用其他registers，必須查simulator table。
2. 在題目BD=0且EPC指向syscall的前提下，服務完成應從下一條instruction繼續，所以saved EPC加4。這是handler的政策，不是MIPS進入例外時自動完成服務的證明。若BD=1，EPC指前一條branch，不能盲目加4後把delay slot當普通順序繼續；需依branch及slot語意恢復。若simulator已代為advance，也不可重複增加。
3. Arithmetic overflow 表示 faulting instruction 的 mathematical result不能合法commit。直接 EPC+4 會跳過 fault 並留下錯誤/未定 result。Handler 必須終止program、模擬替代結果或修正 operand/context，再決定是否 reschedule instruction。
4. Exception entry 可 overwrite caller-visible `$v0/$a0/$t*` 等；不保存會使 interrupted function return with wrong registers。Nested interrupts 可再次改寫同樣 registers，導致 outer handler 或 application state corruption。Save/restore order、stack limit 與 reentrancy 都是必要條件。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

GB-Q07

1. Possible expansions：

```
move $s0,$s1   -> addu $s0,$s1,$zero
la   $t0,label -> lui $t0,hi16(label)
                  ori $t0,$t0,lo16(label)
```

`la` 也可能用 PC-relative sequence 或 linker-relocated pair，依 ISA/toolchain/PIC mode。

2. Absolute literal address 可 encode high/low pair；position-independent code 應由 PC/got 取得，避免 loader relocation。若 label address fit signed 16-bit，assembler 可用 `addiu`，否則需 multi-instruction sequence。
3. SPIM print-string convention：`$v0=4`，`$a0` 放 null-terminated string address。
4. Print integer：`$v0=1`、`$a0=integer`。Exit：`$v0=10`。Source 一行可展開成多條 instructions，或 fit shift/add immediate 等 cases；binary size 由 expansion、alignment、relaxation 與 linker/delay-slot filling 決定。

Appendix B 解答 （續）

GB-Q08

1. P.text到0x00401024為止，Q向上對齊16-byte，故Q.text=0x00401030，padding12 bytes；Q.text尾端0x00401044。P.data尾端0x10010005，Q.data=0x10010008，padding3 bytes，尾端0x1001000c。尾端均為exclusive end，最後一byte還要減1。
2. Foo=0x00401034、item=0x10010008。Jal site=0x00401008，field=0x00401034>>2=0x0010040d，word=0x0c10040d。Ra=site+8=0x00401010；site+4的slot仍先執行。
3. Item=0x10010008時，low=0x0008、high=0x1001。若V=0x1001fffc，low=0xfffc代表−4，high=((V+0x8000)>>16)&0xffff=0x1002；0x10020000−4=0x1001fffc。若仍用high0x1001，則得到0x1000fffc，差0x10000。
4. CPU重建J-target時保留PC+4高4 bits，field只攜帶低28-bit位址中的bits27:2。不同高4-bit區域不能由此欄位表示，截斷會跳到錯區；linker必須報範圍錯或採合法間接序列/veneer，不能把錯誤靜默隱藏。

GB-Q09

1. Main納入後U={f}。LibA抽A1.o滿足f，新增g；U={g}，A2只定義h，當下未被需要，所以不抽。LibB抽B1.o滿足g，新增h；U={h}。流程已越過libA，不自動回頭，故失敗。
2. 再掃一次libA時h未解，抽A2.o後U={}，成功。重列或支援group/rescan是此類archive相依的處理方式；實際選項依linker。
3. 若A1/A2是同一member，f需要它時整個member都被納入，包括h定義。之後B1.o引用h已可解析，原順序成功。抽取單位是object member，不是只抽某個symbol的單獨幾bytes。

Appendix B 解答 （續）

GB-Q10

1. n=2本層sp0x1ff0，saved n在0x1ff8、ra在0x1ffc，return3。n=1本層sp0x1fe0，n在0x1fe8、ra在0x1fec，return1。n=0本層sp0x1fd0，n在0x1fd8、ra在0x1fdc，return0。最多48 bytes，最後sp回0x2000。
2. t0是caller-saved。假設外層n=2先令t0=2，子呼叫n=1返回前把t0改成1，外層再使用t0便算1+1=2而非3。外層的frame裡仍保存正確n，故原程式在call後重新load；不能要求callee無條件保留t0。
3. Delay slot在進入callee之前執行，此時v0尚不是sum_to(n−1)的新結果；而子呼叫之後又會覆寫v0。把相加提前既讀錯時點，也不保證相加結果留存。
4. 需要n為非負整數、n(n+1)/2不超過signed32-bit上限、stack足夠且初始alignment合法。契約為回傳該和並還原sp與callee-saved狀態。Base n=0成立；n>0用已保存的本層n與child契約得到(n−1)n/2+n，epilogue還原本層ra/sp。每次n減1所以終止。有限數例只核對實作軌跡，不取代歸納。

GB-Q11

1. BD=1，EPC=0x00400040。Page fault修復後需使branch與slot語意正確地重試；若只加4變成0x00400044，會丟失branch的控制決策。實際handler需依ISA與fault種類保存/重建足夠狀態，不能一律跳過fault。
2. 明定已完成服務且BD=0，順序返回0x00400084。此處服務不應重做；第1小題則尚未完成load，兩種恢復政策不同。若simulator已自動advance PC，還要避免重複advance，但本題明定由handler選擇。
3. Control讀3次，data只讀1次，最後v0=0x141&0xff=0x41。若data load放在beq的slot，ready=0時仍會執行，可能消耗尚未有效的資料或改變receiver狀態，破壞「ready後才讀且只消耗一次」的協定。Slot nop不是無意義裝飾，而是保留此裝置語意。

Appendix B 解答 （續）

原創驗算程式附錄: assembly_toolchain.py

```
"""Small assembler, relocation, ABI, and exception teaching model."""

from __future__ import annotations

import ast
import re


LABEL = re.compile(r"^\s*([A-Za-z_.$][\w.$]*):\s*(.*)$")


def first_pass_symbols(source: str, text_base: int = 0x00400000, data_base: int = 0x10010000)
-> dict[str, int]:
    """Assign labels for explicit-align data and single-word instructions."""
    symbols: dict[str, int] = {}
    section = "text"
    locations = {"text": text_base, "data": data_base}
    for raw in source.splitlines():
        line = raw.split("#", 1)[0].strip()
        if not line:
            continue
        location = locations[section]
        label_match = LABEL.match(line)
        if label_match:
            symbols[label_match.group(1)] = location
            line = label_match.group(2).strip()
            if not line:
                continue
        parts = line.split(None, 1)
        directive = parts[0].lower()
        if directive == ".text":
            section = "text"
            continue
        if directive == ".data":
```

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Appendix B 解答 （續）

```
            section = "data"
            continue
        if directive == ".align":
            exponent = int(parts[1])
            alignment = 1 << exponent
            location = (location + alignment - 1) & -alignment
        elif directive == ".word":
            location += 4
        elif directive == ".asciiz":
            literal = ast.literal_eval(parts[1].strip())
            location += len(literal.encode("utf-8")) + 1
        elif section == "text":
            location += 4
        else:
            raise ValueError(f"unsupported data directive: {line}")
        locations[section] = location
    return symbols


def j_target_field(target: int) -> int:
    return (target >> 2) & 0x03FFFFFF


def jal_word(target: int) -> int:
    return (3 << 26) | j_target_field(target)


def branch_word_index(pc: int, target: int) -> int:
    delta = target - (pc + 4)
    if delta % 4:
        raise ValueError("unaligned branch target")
    value = delta // 4
    if not -32768 <= value <= 32767:
```

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Appendix B 解答 （續）

```
        raise ValueError("branch target outside 16-bit range")
    return value & 0xFFFF


def load_addiu_constant_halves(value: int) -> tuple[int, int]:
    value &= 0xFFFFFFFF
    low = value & 0xFFFF
    high = ((value + 0x8000) >> 16) & 0xFFFF
    return high, low


def main() -> None:
    source = """
    .text
    main: addiu $t0, $zero, 0
    loop: addiu $t0, $t0, 1
          beq   $t0, $a0, loop
          nop
    done: jr    $ra
          nop
    func: addiu $v0, $zero, 7
          jr    $ra
          nop
    .data
    .align 2
    table: .word 0xdeadbeef
    count: .word 3
    str:   .asciiz "Hi"
    """
    symbols = first_pass_symbols(source)
    assert symbols == {
        "main": 0x00400000,
        "loop": 0x00400004,
```

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Appendix B 解答 （續）

```
        "done": 0x00400010,
        "func": 0x00400018,
        "table": 0x10010000,
        "count": 0x10010004,
        "str": 0x10010008,
    }
    assert branch_word_index(0x00400008, symbols["loop"]) == 0xFFFE
    assert j_target_field(symbols["done"]) == 0x00100004
    assert jal_word(0x00402000) == 0x0C100800
    assert j_target_field(symbols["func"]) == 0x00100006
    assert first_pass_symbols('.text\na: nop\n.data\nx: .word 1\n.text\nb: nop') == {"a":
0x00400000, "x": 0x10010000, "b": 0x00400004}
    for value in (0x12345678, 0x1234ABCD, 0xFFFF8000):
        high, low = load_addiu_constant_halves(value)
        signed_low = low if low < 0x8000 else low - 0x10000
        assert ((high << 16) + signed_low) & 0xFFFFFFFF == value
        assert (((value >> 16) << 16) | low) == value
    assert 4 * 16 == 64
    assert (0x00400000 + 192) == 0x004000C0
    assert (0x10010010 + 24) == 0x10010028

    print("symbols:", " ".join(f"{name}=0x{value:08x}" for name, value in symbols.items()))
    print(f"beq immediate: 0x{branch_word_index(0x00400008, symbols['loop']):04x}")
    print(f"jal word: 0x{jal_word(0x00402000):08x}")
    print("lui/addiu adjusted halves:", tuple(hex(x) for x in
load_addiu_constant_halves(0x1234ABCD)))
    print("sum_to(3) stack bytes:", 4 * 16)
    print("toolchain checks: PASS")


if __name__ == "__main__":
    main()
```

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

參考資料與署名

David A. Patterson, John L. Hennessy. Computer Organization and Design: The Hardware/Software Interface, Revised Fourth Edition.

Lin Cing 原創教學整理。原創內容以 CC BY 4.0 授權；參考書籍不隨教材散布。

https://github.com/xinhi6716/computer-science-self-study